

lab1实验报告

思考题

Thinking 1.1

Thinking 1.1 在阅读 附录中的编译链接详解 以及本章内容后，尝试分别使用实验环境中 的原生 x86 工具链 (gcc、ld、readelf、objdump 等) 和 MIPS 交叉编译工具链 (带有 mips-linux-gnu-前缀，如 mips-linux-gnu-gcc、mips-linux-gnu-ld)，重复其中的编译和解析过程，观察相应的结果，并解释其中向 objdump 传入的参数的含义。

编写一个简单的C语言程序如下

```
1 // main.c
2 #include<stdio.h>
3 int main() {
4     printf("hello,world!\n");
5     return 0;
6 }
```

分别调用x86和交叉编译器来编译该文件

```
1 gcc -o main_x86 main.c
2 mips-linux-gnu-gcc -o main_mips hello.c
```

对编译结果解析之后可以发现两者不同，最直观的不同是反汇编之后出现了不同架构的汇编代码。

一、解析main_x86

readelf

执行 `readelf -a main_x86` 得到以下结果

```
1 ELF 头:
2   Magic:      7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00
3   类别:                                ELF64
4   数据:                                2 补码, 小端序 (little endian)
5   Version:                                1 (current)
6   OS/ABI:                                UNIX - System V
7   ABI 版本:                                0
8   类型:                                DYN (Position-Independent Executable
file)
9   系统架构:                                Advanced Micro Devices x86-64
10  版本:                                0x1
11  入口点地址:                                0x1060
12  程序头起点:                                64 (bytes into file)
13  Start of section headers:                13976 (bytes into file)
14  标志:                                0x0
15  Size of this header:                    64 (bytes)
16  Size of program headers:                56 (bytes)
17  Number of program headers:                13
18  Size of section headers:                64 (bytes)
```

```

19 Number of section headers: 31
20 Section header string table index: 30
21
22 节头:
23 [号] 名称 类型 地址 偏移量
24 大小 全体大小 旗标 链接 信息 对齐
25 [ 0] NULL 0000000000000000 00000000
26 0000000000000000 0000000000000000 0 0 0
27 [ 1] .interp PROGBITS 0000000000000318 00000318
28 000000000000001c 0000000000000000 A 0 0 1
29 [ 2] .note.gnu.pr[...] NOTE 0000000000000338 00000338
30 0000000000000030 0000000000000000 A 0 0 8
31 [ 3] .note.gnu.bu[...] NOTE 0000000000000368 00000368
32 0000000000000024 0000000000000000 A 0 0 4
33 [ 4] .note.ABI-tag NOTE 000000000000038c 0000038c
34 0000000000000020 0000000000000000 A 0 0 4
35 [ 5] .gnu.hash GNU_HASH 00000000000003b0 000003b0
36 0000000000000024 0000000000000000 A 6 0 8
37 [ 6] .dynsym DYSYM 00000000000003d8 000003d8
38 00000000000000a8 0000000000000018 A 7 1 8
39 [ 7] .dynstr STRTAB 0000000000000480 00000480
40 000000000000008d 0000000000000000 A 0 0 1
41 [ 8] .gnu.version VERSYM 000000000000050e 0000050e
42 000000000000000e 0000000000000002 A 6 0 2
43 [ 9] .gnu.version_r VERNEED 0000000000000520 00000520
44 0000000000000030 0000000000000000 A 7 1 8
45 [10] .rela.dyn RELA 0000000000000550 00000550
46 00000000000000c0 0000000000000018 A 6 0 8
47 [11] .rela.plt RELA 0000000000000610 00000610
48 0000000000000018 0000000000000018 AI 6 24 8
49 [12] .init PROGBITS 0000000000001000 00001000
50 000000000000001b 0000000000000000 AX 0 0 4
51 [13] .plt PROGBITS 0000000000001020 00001020
52 0000000000000020 0000000000000010 AX 0 0 16
53 [14] .plt.got PROGBITS 0000000000001040 00001040
54 0000000000000010 0000000000000010 AX 0 0 16
55 [15] .plt.sec PROGBITS 0000000000001050 00001050
56 0000000000000010 0000000000000010 AX 0 0 16
57 [16] .text PROGBITS 0000000000001060 00001060
58 0000000000000107 0000000000000000 AX 0 0 16
59 [17] .fini PROGBITS 0000000000001168 00001168
60 000000000000000d 0000000000000000 AX 0 0 4
61 [18] .rodata PROGBITS 0000000000002000 00002000
62 0000000000000011 0000000000000000 A 0 0 4
63 [19] .eh_frame_hdr PROGBITS 0000000000002014 00002014
64 0000000000000034 0000000000000000 A 0 0 4
65 [20] .eh_frame PROGBITS 0000000000002048 00002048
66 00000000000000ac 0000000000000000 A 0 0 8
67 [21] .init_array INIT_ARRAY 0000000000003db8 00002db8
68 0000000000000008 0000000000000008 WA 0 0 8
69 [22] .fini_array FINI_ARRAY 0000000000003dc0 00002dc0
70 0000000000000008 0000000000000008 WA 0 0 8
71 [23] .dynamic DYNAMIC 0000000000003dc8 00002dc8
72 00000000000001f0 0000000000000010 WA 7 0 8
73 [24] .got PROGBITS 0000000000003fb8 00002fb8
74 0000000000000048 0000000000000008 WA 0 0 8
75 [25] .data PROGBITS 0000000000004000 00003000
76 0000000000000010 0000000000000000 WA 0 0 8

```

```

77 [26] .bss NOBITS 0000000000004010 00003010
78 0000000000000008 0000000000000000 WA 0 0 1
79 [27] .comment PROGBITS 0000000000000000 00003010
80 000000000000002b 0000000000000001 MS 0 0 1
81 [28] .symtab SYMTAB 0000000000000000 00003040
82 0000000000000360 0000000000000018 29 18 8
83 [29] .strtab STRTAB 0000000000000000 000033a0
84 00000000000001da 0000000000000000 0 0 1
85 [30] .shstrtab STRTAB 0000000000000000 0000357a
86 000000000000011a 0000000000000000 0 0 1

```

Key to Flags:

```

88 W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
89 L (link order), O (extra OS processing required), G (group), T (TLS),
90 C (compressed), x (unknown), o (OS specific), E (exclude),
91 D (mbind), l (large), p (processor specific)

```

There are no section groups in this file.

程序头:

Type	Offset FileSiz	VirtAddr MemSiz	PhysAddr Flags Align
PHDR	0x0000000000000040	0x0000000000000040	0x0000000000000040
	0x00000000000002d8	0x00000000000002d8	R 0x8
INTERP	0x0000000000000318	0x0000000000000318	0x0000000000000318
	0x000000000000001c	0x000000000000001c	R 0x1
[Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]			
LOAD	0x0000000000000000	0x0000000000000000	0x0000000000000000
	0x0000000000000628	0x0000000000000628	R 0x1000
LOAD	0x0000000000000100	0x0000000000000100	0x0000000000000100
	0x0000000000000175	0x0000000000000175	R E 0x1000
LOAD	0x0000000000000200	0x0000000000000200	0x0000000000000200
	0x00000000000000f4	0x00000000000000f4	R 0x1000
LOAD	0x00000000000002db8	0x00000000000003db8	0x00000000000003db8
	0x0000000000000258	0x0000000000000260	RW 0x1000
DYNAMIC	0x00000000000002dc8	0x00000000000003dc8	0x00000000000003dc8
	0x000000000000001f0	0x000000000000001f0	RW 0x8
NOTE	0x00000000000000338	0x00000000000000338	0x00000000000000338
	0x00000000000000030	0x00000000000000030	R 0x8
NOTE	0x00000000000000368	0x00000000000000368	0x00000000000000368
	0x00000000000000044	0x00000000000000044	R 0x4
GNU_PROPERTY	0x00000000000000338	0x00000000000000338	0x00000000000000338
	0x00000000000000030	0x00000000000000030	R 0x8
GNU_EH_FRAME	0x00000000000002014	0x00000000000002014	0x00000000000002014
	0x00000000000000034	0x00000000000000034	R 0x4
GNU_STACK	0x00000000000000000	0x00000000000000000	0x00000000000000000
	0x00000000000000000	0x00000000000000000	RW 0x10
GNU_RELRO	0x00000000000002db8	0x00000000000003db8	0x00000000000003db8
	0x00000000000000248	0x00000000000000248	R 0x1

Section to Segment mapping:

段节...

```

128 00
129 01 .interp
130 02 .interp .note.gnu.property .note.gnu.build-id .note.ABI-tag
.gnu.hash .dynsym .dynstr .gnu.version .gnu.version_r .rela.dyn .rela.plt
131 03 .init.plt .plt.got .plt.sec .text .fini
132 04 .rodata .eh_frame_hdr .eh_frame
133 05 .init_array .fini_array .dynamic .got .data .bss

```

```

134 06 .dynamic
135 07 .note.gnu.property
136 08 .note.gnu.build-id .note.ABI-tag
137 09 .note.gnu.property
138 10 .eh_frame_hdr
139 11
140 12 .init_array .fini_array .dynamic .got
141

```

142 Dynamic section at offset 0x2dc8 contains 27 entries:

标记	类型	名称/值
0x0000000000000001	(NEEDED)	共享库: [libc.so.6]
0x000000000000000c	(INIT)	0x1000
0x000000000000000d	(FINI)	0x1168
0x0000000000000019	(INIT_ARRAY)	0x3db8
0x000000000000001b	(INIT_ARRAYSZ)	8 (bytes)
0x000000000000001a	(FINI_ARRAY)	0x3dc0
0x000000000000001c	(FINI_ARRAYSZ)	8 (bytes)
0x000000006ffffef5	(GNU_HASH)	0x3b0
0x0000000000000005	(STRTAB)	0x480
0x0000000000000006	(SYMTAB)	0x3d8
0x000000000000000a	(STRSZ)	141 (bytes)
0x000000000000000b	(SYMENT)	24 (bytes)
0x0000000000000015	(DEBUG)	0x0
0x0000000000000003	(PLTGOT)	0x3fb8
0x0000000000000002	(PLTRELSZ)	24 (bytes)
0x0000000000000014	(PLTREL)	RELA
0x0000000000000017	(JMPREL)	0x610
0x0000000000000007	(RELA)	0x550
0x0000000000000008	(RELASZ)	192 (bytes)
0x0000000000000009	(RELAENT)	24 (bytes)
0x000000000000001e	(FLAGS)	BIND_NOW
0x000000006fffffb	(FLAGS_1)	标志: NOW PIE
0x000000006fffffe	(VERNEED)	0x520
0x000000006ffffff	(VERNEEDNUM)	1
0x000000006fffff0	(VERSYM)	0x50e
0x000000006fffff9	(RELACOUNT)	3
0x0000000000000000	(NULL)	0x0

172 重定位节 '.rela.dyn' at offset 0x550 contains 8 entries:

偏移量	信息	类型	符号值	符号名称 + 加数
000000003db8	000000000008	R_X86_64_RELATIVE		1140
000000003dc0	000000000008	R_X86_64_RELATIVE		1100
000000004008	000000000008	R_X86_64_RELATIVE		4008
000000003fd8	000100000006	R_X86_64_GLOB_DAT	0000000000000000	__libc_start_main@GLIBC_2.34 + 0
000000003fe0	000200000006	R_X86_64_GLOB_DAT	0000000000000000	_ITM_deregisterTM[...] + 0
000000003fe8	000400000006	R_X86_64_GLOB_DAT	0000000000000000	__gmon_start__ + 0
000000003ff0	000500000006	R_X86_64_GLOB_DAT	0000000000000000	_ITM_registerTMCI[...] + 0
000000003ff8	000600000006	R_X86_64_GLOB_DAT	0000000000000000	__cxa_finalize@GLIBC_2.2.5 + 0

183 重定位节 '.rela.plt' at offset 0x610 contains 1 entry:

偏移量	信息	类型	符号值	符号名称 + 加数
000000003fd0	000300000007	R_X86_64_JUMP_SLO	0000000000000000	puts@GLIBC_2.2.5 + 0

```

186 No processor specific unwind information to decode
187
188 Symbol table '.dynsym' contains 7 entries:
189   Num:   Value                               Size Type   Bind    Vis      Ndx Name
190     0: 0000000000000000      0 NOTYPE  LOCAL  DEFAULT  UND
191     1: 0000000000000000      0 FUNC    GLOBAL  DEFAULT  UND
192   _[...]@GLIBC_2.34 (2)
193     2: 0000000000000000      0 NOTYPE  WEAK    DEFAULT  UND
194   _ITM_deregisterT[...]
195     3: 0000000000000000      0 FUNC    GLOBAL  DEFAULT  UND
196   puts@GLIBC_2.2.5 (3)
197     4: 0000000000000000      0 NOTYPE  WEAK    DEFAULT  UND __gmon_start__
198     5: 0000000000000000      0 NOTYPE  WEAK    DEFAULT  UND
199   _ITM_registerTMC[...]
200     6: 0000000000000000      0 FUNC    WEAK    DEFAULT  UND
201   [...]@GLIBC_2.2.5 (3)
202
203 Symbol table '.symtab' contains 36 entries:
204   Num:   Value                               Size Type   Bind    Vis      Ndx Name
205     0: 0000000000000000      0 NOTYPE  LOCAL  DEFAULT  UND
206     1: 0000000000000000      0 FILE    LOCAL  DEFAULT  ABS Scrt1.o
207     2: 000000000000038c     32 OBJECT  LOCAL  DEFAULT    4 __abi_tag
208     3: 0000000000000000      0 FILE    LOCAL  DEFAULT  ABS crtstuff.c
209     4: 0000000000001090      0 FUNC    LOCAL  DEFAULT   16
210   deregister_tm_clones
211     5: 00000000000010c0      0 FUNC    LOCAL  DEFAULT   16
212   register_tm_clones
213     6: 0000000000001100      0 FUNC    LOCAL  DEFAULT   16
214   __do_global_ctors_aux
215     7: 0000000000004010      1 OBJECT  LOCAL  DEFAULT   26 completed.0
216     8: 0000000000003dc0      0 OBJECT  LOCAL  DEFAULT   22
217   __do_global_dtor[...]
218     9: 0000000000001140      0 FUNC    LOCAL  DEFAULT   16 frame_dummy
219    10: 0000000000003db8      0 OBJECT  LOCAL  DEFAULT   21
220   __frame_dummy_in[...]
221    11: 0000000000000000      0 FILE    LOCAL  DEFAULT  ABS main.c
222    12: 0000000000000000      0 FILE    LOCAL  DEFAULT  ABS crtstuff.c
223    13: 00000000000020f0      0 OBJECT  LOCAL  DEFAULT   20 __FRAME_END__
224    14: 0000000000000000      0 FILE    LOCAL  DEFAULT  ABS
225    15: 0000000000003dc8      0 OBJECT  LOCAL  DEFAULT   23 _DYNAMIC
226    16: 0000000000002014      0 NOTYPE  LOCAL  DEFAULT   19
227   __GNU_EH_FRAME_HDR
228    17: 0000000000003fb8      0 OBJECT  LOCAL  DEFAULT   24
229   _GLOBAL_OFFSET_TABLE_
230    18: 0000000000000000      0 FUNC    GLOBAL  DEFAULT  UND
231   __libc_start_mai[...]
232    19: 0000000000000000      0 NOTYPE  WEAK    DEFAULT  UND
233   _ITM_deregisterT[...]
234    20: 0000000000004000      0 NOTYPE  WEAK    DEFAULT   25 data_start
235    21: 0000000000000000      0 FUNC    GLOBAL  DEFAULT  UND
236   puts@GLIBC_2.2.5
237    22: 0000000000004010      0 NOTYPE  GLOBAL  DEFAULT   25 _edata
238    23: 0000000000001168      0 FUNC    GLOBAL  HIDDEN   17 _fini
239    24: 0000000000004000      0 NOTYPE  GLOBAL  DEFAULT   25 __data_start
240    25: 0000000000000000      0 NOTYPE  WEAK    DEFAULT  UND __gmon_start__
241    26: 0000000000004008      0 OBJECT  GLOBAL  HIDDEN   25 __dso_handle
242    27: 0000000000002000      4 OBJECT  GLOBAL  DEFAULT   18 _IO_stdin_used
243    28: 0000000000004018      0 NOTYPE  GLOBAL  DEFAULT   26 _end

```

```

229      29: 0000000000001060      38 FUNC      GLOBAL DEFAULT   16 _start
230      30: 0000000000004010       0 NOTYPE     GLOBAL DEFAULT   26 __bss_start
231      31: 0000000000001149      30 FUNC      GLOBAL DEFAULT   16 main
232      32: 0000000000004010       0 OBJECT     GLOBAL HIDDEN    25 __TMC_END__
233      33: 0000000000000000       0 NOTYPE     WEAK      DEFAULT   UND
    _ITM_registerTMC[...]
234      34: 0000000000000000       0 FUNC      WEAK      DEFAULT   UND
    __cxa_finalize@G[...]
235      35: 0000000000001000       0 FUNC      GLOBAL HIDDEN    12 _init
236
237 Histogram for '.gnu.hash' bucket list length (total of 2 buckets):
238   Length  Number      % of total  Coverage
239       0    1          ( 50.0%)
240       1    1          ( 50.0%)   100.0%
241
242 Version symbols section '.gnu.version' contains 7 entries:
243   Addr: 0x000000000000050e  Offset: 0x0000050e  Link: 6 (.dynsym)
244   000:   0 (*本地*)         2 (GLIBC_2.34)       1 (*全局*)         3 (GLIBC_2.2.5)
245   004:   1 (*全局*)         1 (*全局*)         3 (GLIBC_2.2.5)
246
247 Version needs section '.gnu.version_r' contains 1 entry:
248   Addr: 0x0000000000000520  Offset: 0x00000520  Link: 7 (.dynstr)
249   000000: Version: 1  文件: libc.so.6  计数: 2
250   0x0010:   Name: GLIBC_2.2.5  标志: 无  版本: 3
251   0x0020:   Name: GLIBC_2.34  标志: 无  版本: 2
252
253 Displaying notes found in: .note.gnu.property
254   所有者          Data size  Description
255   GNU              0x00000020      NT_GNU_PROPERTY_TYPE_0
256   Properties: x86 feature: IBT, SHSTK
257   x86 ISA needed: x86-64-baseline
258
259 Displaying notes found in: .note.gnu.build-id
260   所有者          Data size  Description
261   GNU              0x00000014      NT_GNU_BUILD_ID (unique build ID
    bitstring)
262   Build ID: 5aa0972205128976248014c187fa1d21c7b705f8
263
264 Displaying notes found in: .note.ABI-tag
265   所有者          Data size  Description
266   GNU              0x00000010      NT_GNU_ABI_TAG (ABI version tag)
267   OS: Linux, ABI: 3.2.0

```

objdump

执行 `objdump -S main_x86` 得到以下结果

```

1  main_x86:      文件格式 elf64-x86-64
2
3
4  Disassembly of section .init:
5
6  0000000000001000 <_init>:
7      1000:      f3 0f 1e fa                endbr64
8      1004:      48 83 ec 08                sub     $0x8,%rsp
9      1008:      48 8b 05 d9 2f 00 00      mov     0x2fd9(%rip),%rax      #
    3fe8 <__gmon_start__@Base>

```

```

10      100f:      48 85 c0          test    %rax,%rax
11      1012:      74 02          je      1016 <_init+0x16>
12      1014:      ff d0          call    *%rax
13      1016:      48 83 c4 08      add     $0x8,%rsp
14      101a:      c3              ret
15
16  Disassembly of section .plt:
17
18  0000000000001020 <.plt>:
19      1020:      ff 35 9a 2f 00 00      push    0x2f9a(%rip)          # 3fc0
    <_GLOBAL_OFFSET_TABLE_+0x8>
20      1026:      ff 25 9c 2f 00 00      jmp     *0x2f9c(%rip)          # 3fc8
    <_GLOBAL_OFFSET_TABLE_+0x10>
21      102c:      0f 1f 40 00          nopl    0x0(%rax)
22      1030:      f3 0f 1e fa          endbr64
23      1034:      68 00 00 00 00      push    $0x0
24      1039:      e9 e2 ff ff ff      jmp     1020 <_init+0x20>
25      103e:      66 90              xchg    %ax,%ax
26
27  Disassembly of section .plt.got:
28
29  0000000000001040 <__cxa_finalize@plt>:
30      1040:      f3 0f 1e fa          endbr64
31      1044:      ff 25 ae 2f 00 00      jmp     *0x2fae(%rip)          # 3ff8
    <__cxa_finalize@GLIBC_2.2.5>
32      104a:      66 0f 1f 44 00 00      nopw    0x0(%rax,%rax,1)
33
34  Disassembly of section .plt.sec:
35
36  0000000000001050 <puts@plt>:
37      1050:      f3 0f 1e fa          endbr64
38      1054:      ff 25 76 2f 00 00      jmp     *0x2f76(%rip)          # 3fd0
    <puts@GLIBC_2.2.5>
39      105a:      66 0f 1f 44 00 00      nopw    0x0(%rax,%rax,1)
40
41  Disassembly of section .text:
42
43  0000000000001060 <_start>:
44      1060:      f3 0f 1e fa          endbr64
45      1064:      31 ed              xor     %ebp,%ebp
46      1066:      49 89 d1          mov     %rdx,%r9
47      1069:      5e              pop     %rsi
48      106a:      48 89 e2          mov     %rsp,%rdx
49      106d:      48 83 e4 f0      and     $0xfffffffffffffff0,%rsp
50      1071:      50              push    %rax
51      1072:      54              push    %rsp
52      1073:      45 31 c0          xor     %r8d,%r8d
53      1076:      31 c9              xor     %ecx,%ecx
54      1078:      48 8d 3d ca 00 00 00      lea     0xca(%rip),%rdi          #
1149 <main>
55      107f:      ff 15 53 2f 00 00      call    *0x2f53(%rip)          # 3fd8
    <__libc_start_main@GLIBC_2.34>
56      1085:      f4              hlt
57      1086:      66 2e 0f 1f 84 00 00      cs nopw 0x0(%rax,%rax,1)
58      108d:      00 00 00
59
60  0000000000001090 <deregister_tm_clones>:

```

```

61      1090:      48 8d 3d 79 2f 00 00    lea     0x2f79(%rip),%rdi      #
4010 <__TMC_END__>
62      1097:      48 8d 05 72 2f 00 00    lea     0x2f72(%rip),%rax      #
4010 <__TMC_END__>
63      109e:      48 39 f8                cmp     %rdi,%rax
64      10a1:      74 15                    je      10b8
<deregister_tm_clones+0x28>
65      10a3:      48 8b 05 36 2f 00 00    mov     0x2f36(%rip),%rax      #
3fe0 <__ITM_deregisterTMCloneTable@Base>
66      10aa:      48 85 c0                test    %rax,%rax
67      10ad:      74 09                    je      10b8
<deregister_tm_clones+0x28>
68      10af:      ff e0                    jmp     *%rax
69      10b1:      0f 1f 80 00 00 00 00    nopl    0x0(%rax)
70      10b8:      c3                      ret
71      10b9:      0f 1f 80 00 00 00 00    nopl    0x0(%rax)
72
73      000000000000010c0 <register_tm_clones>:
74      10c0:      48 8d 3d 49 2f 00 00    lea     0x2f49(%rip),%rdi      #
4010 <__TMC_END__>
75      10c7:      48 8d 35 42 2f 00 00    lea     0x2f42(%rip),%rsi      #
4010 <__TMC_END__>
76      10ce:      48 29 fe                sub     %rdi,%rsi
77      10d1:      48 89 f0                mov     %rsi,%rax
78      10d4:      48 c1 ee 3f            shr     $0x3f,%rsi
79      10d8:      48 c1 f8 03            sar     $0x3,%rax
80      10dc:      48 01 c6                add     %rax,%rsi
81      10df:      48 d1 fe                sar     $1,%rsi
82      10e2:      74 14                    je      10f8
<register_tm_clones+0x38>
83      10e4:      48 8b 05 05 2f 00 00    mov     0x2f05(%rip),%rax      #
3ff0 <__ITM_registerTMCloneTable@Base>
84      10eb:      48 85 c0                test    %rax,%rax
85      10ee:      74 08                    je      10f8
<register_tm_clones+0x38>
86      10f0:      ff e0                    jmp     *%rax
87      10f2:      66 0f 1f 44 00 00      nopw    0x0(%rax,%rax,1)
88      10f8:      c3                      ret
89      10f9:      0f 1f 80 00 00 00 00    nopl    0x0(%rax)
90
91      00000000000001100 <__do_global_dtors_aux>:
92      1100:      f3 0f 1e fa            endbr64
93      1104:      80 3d 05 2f 00 00 00    cmpb    $0x0,0x2f05(%rip)      #
4010 <__TMC_END__>
94      110b:      75 2b                    jne     1138
<__do_global_dtors_aux+0x38>
95      110d:      55                      push    %rbp
96      110e:      48 83 3d e2 2e 00 00    cmpq    $0x0,0x2ee2(%rip)      #
3ff8 <__cxa_finalize@GLIBC_2.2.5>
97      1115:      00                      mov     %rsp,%rbp
98      1116:      48 89 e5                mov     %rsp,%rbp
99      1119:      74 0c                    je      1127
<__do_global_dtors_aux+0x27>
100     111b:      48 8b 3d e6 2e 00 00    mov     0x2ee6(%rip),%rdi      #
4008 <__dso_handle>
101     1122:      e8 19 ff ff ff        call    1040 <__cxa_finalize@plt>
102     1127:      e8 64 ff ff ff        call    1090 <deregister_tm_clones>

```


103	112c:	c6 05 dd 2e 00 00 01	movb	\$0x1,0x2edd(%rip)	#
	4010 <__TMC_END__>				
104	1133:	5d	pop	%rbp	
105	1134:	c3	ret		
106	1135:	0f 1f 00	nopl	(%rax)	
107	1138:	c3	ret		
108	1139:	0f 1f 80 00 00 00 00	nopl	0x0(%rax)	
109					
110	0000000000001140 <frame_dummy>:				
111	1140:	f3 0f 1e fa	endbr64		
112	1144:	e9 77 ff ff ff	jmp	10c0 <register_tm_clones>	
113					
114	0000000000001149 <main>:				
115	1149:	f3 0f 1e fa	endbr64		
116	114d:	55	push	%rbp	
117	114e:	48 89 e5	mov	%rsp,%rbp	
118	1151:	48 8d 05 ac 0e 00 00	lea	0xeac(%rip),%rax	#
	2004 <_IO_stdin_used+0x4>				
119	1158:	48 89 c7	mov	%rax,%rdi	
120	115b:	e8 f0 fe ff ff	call	1050 <puts@plt>	
121	1160:	b8 00 00 00 00	mov	\$0x0,%eax	
122	1165:	5d	pop	%rbp	
123	1166:	c3	ret		
124					
125	Disassembly of section .fini:				
126					
127	0000000000001168 <_fini>:				
128	1168:	f3 0f 1e fa	endbr64		
129	116c:	48 83 ec 08	sub	\$0x8,%rsp	
130	1170:	48 83 c4 08	add	\$0x8,%rsp	
131	1174:	c3	ret		

二、解析main_mips

readelf

执行 `mips-linux-gnu-readelf -a main_mips` 得到以下结果

1	ELF 头:	
2	Magic:	7f 45 4c 46 01 02 01 00 00 00 00 00 00 00 00 00
3	类别:	ELF32
4	数据:	2 补码, 大端序 (big endian)
5	Version:	1 (current)
6	OS/ABI:	UNIX - System V
7	ABI 版本:	0
8	类型:	EXEC (可执行文件)
9	系统架构:	MIPS R3000
10	版本:	0x1
11	入口点地址:	0x400520
12	程序头起点:	52 (bytes into file)
13	Start of section headers:	67612 (bytes into file)
14	标志:	0x70001007, noreorder, pic, cpic, o32, mips32r2
15	Size of this header:	52 (bytes)
16	Size of program headers:	32 (bytes)
17	Number of program headers:	11
18	Size of section headers:	40 (bytes)
19	Number of section headers:	32

20 Section header string table index: 31

21
22 节头:

23	[Nr]	Name	Type	Addr	off	Size	ES	Flg	Lk
	Inf	A1							
24	[0]		NULL	00000000	000000	000000	00		0
	0	0							
25	[1]	.interp	PROGBITS	00400194	000194	00000d	00	A	0
	0	1							
26	[2]	.MIPS.abiflags	MIPS_ABIFLAGS	004001a8	0001a8	000018	18	A	0
	0	8							
27	[3]	.reginfo	MIPS_REGINFO	004001c0	0001c0	000018	18	A	0
	0	4							
28	[4]	.note.gnu.bu[...]	NOTE	004001d8	0001d8	000024	00	A	0
	0	4							
29	[5]	.note.ABI-tag	NOTE	004001fc	0001fc	000020	00	A	0
	0	4							
30	[6]	.dynamic	DYNAMIC	0040021c	00021c	000100	08	A	9
	0	4							
31	[7]	.hash	HASH	0040031c	00031c	00003c	04	A	8
	0	4							
32	[8]	.dynsym	DYNSYM	00400358	000358	0000a0	10	A	9
	1	4							
33	[9]	.dynstr	STRTAB	004003f8	0003f8	0000a6	00	A	0
	0	1							
34	[10]	.gnu.version	VERSYM	0040049e	00049e	000014	02	A	8
	0	2							
35	[11]	.gnu.version_r	VERNEED	004004b4	0004b4	000030	00	A	9
	1	4							
36	[12]	.init	PROGBITS	004004e4	0004e4	00003c	00	AX	0
	0	4							
37	[13]	.text	PROGBITS	00400520	000520	000190	00	AX	0
	0	16							
38	[14]	.MIPS.stubs	PROGBITS	004006b0	0006b0	000030	00	AX	0
	0	4							
39	[15]	.fini	PROGBITS	004006e0	0006e0	000024	00	AX	0
	0	4							
40	[16]	.rodata	PROGBITS	00400710	000710	000020	00	A	0
	0	16							
41	[17]	.eh_frame	PROGBITS	00400730	000730	000004	00	A	0
	0	4							
42	[18]	.init_array	INIT_ARRAY	0041fff8	00fff8	000004	04	WA	0
	0	4							
43	[19]	.fini_array	FINI_ARRAY	0041fffc	00fffc	000004	04	WA	0
	0	4							
44	[20]	.data	PROGBITS	00420000	010000	000010	00	WA	0
	0	16							
45	[21]	.rld_map	PROGBITS	00420010	010010	000004	00	WA	0
	0	4							
46	[22]	.got	PROGBITS	00420020	010020	000020	04	WAp	0
	0	16							
47	[23]	.sdata	PROGBITS	00420040	010040	000004	00	WAp	0
	0	4							
48	[24]	.bss	NOBITS	00420050	010044	000010	00	WA	0
	0	16							
49	[25]	.comment	PROGBITS	00000000	010044	000026	01	MS	0
	0	1							

```

50 [26] .pdr PROGBITS 00000000 01006c 000020 00 0
    0 4
51 [27] .gnu.attributes GNU_ATTRIBUTES 00000000 01008c 000010 00 0
    0 1
52 [28] .mdebug.abi32 PROGBITS 00000000 01009c 000000 00 0
    0 1
53 [29] .symtab SYMTAB 00000000 01009c 000460 10 30
    47 4
54 [30] .strtab STRTAB 00000000 0104fc 0001f5 00 0
    0 1
55 [31] .shstrtab STRTAB 00000000 0106f1 00012b 00 0
    0 1

```

56 Key to Flags:

```

57 W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
58 L (link order), O (extra OS processing required), G (group), T (TLS),
59 C (compressed), x (unknown), o (OS specific), E (exclude),
60 D (mbind), p (processor specific)
61

```

62 There are no section groups in this file.

63
64 程序头:

Type	Offset	VirtAddr	PhysAddr	FileSiz	MemSiz	Flg	Align
PHDR	0x000034	0x00400034	0x00400034	0x00160	0x00160	R	0x4
INTERP	0x000194	0x00400194	0x00400194	0x0000d	0x0000d	R	0x1
[Requesting program interpreter: /lib/ld.so.1]							
ABIFLAGS	0x0001a8	0x004001a8	0x004001a8	0x00018	0x00018	R	0x8
REGINFO	0x0001c0	0x004001c0	0x004001c0	0x00018	0x00018	R	0x4
LOAD	0x000000	0x00400000	0x00400000	0x00734	0x00734	R E	
	0x10000						
LOAD	0x00fff8	0x0041fff8	0x0041fff8	0x0004c	0x00068	RW	
	0x10000						
DYNAMIC	0x00021c	0x0040021c	0x0040021c	0x00100	0x00100	R	0x4
NOTE	0x0001d8	0x004001d8	0x004001d8	0x00044	0x00044	R	0x4
GNU_STACK	0x000000	0x00000000	0x00000000	0x00000	0x00000	RWE	0x10
GNU_RELRO	0x00fff8	0x0041fff8	0x0041fff8	0x00008	0x00008	R	0x1
NULL	0x000000	0x00000000	0x00000000	0x00000	0x00000		0x4

78
79 Section to Segment mapping:

80 段节...

```

81 00
82 01 .interp
83 02 .MIPS.abiflags
84 03 .reginfo
85 04 .interp .MIPS.abiflags .reginfo .note.gnu.build-id .note.ABI-tag
    .dynamic .hash .dynsym .dynstr .gnu.version .gnu.version_r .init .text
    .MIPS.stubs .fini .rodata .eh_frame
86 05 .init_array .fini_array .data .rld_map .got .sdata .bss
87 06 .dynamic
88 07 .note.gnu.build-id .note.ABI-tag
89 08
90 09 .init_array .fini_array
91 10
92

```

93 Dynamic section at offset 0x21c contains 27 entries:

标记	类型	名称/值
0x00000001 (NEEDED)		共享库: [libc.so.6]
0x0000000c (INIT)		0x4004e4
0x0000000d (FINI)		0x4006e0

```

98 0x00000019 (INIT_ARRAY) 0x41fff8
99 0x0000001b (INIT_ARRAYSZ) 4 (bytes)
100 0x0000001a (FINI_ARRAY) 0x41fffc
101 0x0000001c (FINI_ARRAYSZ) 4 (bytes)
102 0x00000004 (HASH) 0x40031c
103 0x00000005 (STRTAB) 0x4003f8
104 0x00000006 (SYMTAB) 0x400358
105 0x0000000a (STRSZ) 166 (bytes)
106 0x0000000b (SYMENT) 16 (bytes)
107 0x70000016 (MIPS_RLD_MAP) 0x420010
108 0x70000035 (MIPS_RLD_MAP_REL) 0x1fd8c
109 0x00000015 (DEBUG) 0x0
110 0x00000003 (PLTGOT) 0x420020
111 0x70000001 (MIPS_RLD_VERSION) 1
112 0x70000005 (MIPS_FLAGS) NOTPOT
113 0x70000006 (MIPS_BASE_ADDRESS) 0x400000
114 0x7000000a (MIPS_LOCAL_GOTNO) 3
115 0x70000011 (MIPS_SYMTABNO) 10
116 0x70000012 (MIPS_UNREFEXTNO) 29
117 0x70000013 (MIPS_GOTSYM) 0x5
118 0x6ffffffe (VERNEED) 0x4004b4
119 0x6fffffff (VERNEEDNUM) 1
120 0x6ffffff0 (VERSYM) 0x40049e
121 0x00000000 (NULL) 0x0
122
123 该文件中没有重定位信息。
124
125 The decoding of unwind sections for machine type MIPS R3000 is not
    currently supported.
126
127 Symbol table '.dynsym' contains 10 entries:
128   Num:   Value   Size Type   Bind   Vis     Ndx Name
129   0: 00000000    0 NOTYPE  LOCAL  DEFAULT UND
130   1: 00000001    0 SECTION GLOBAL  DEFAULT ABS _DYNAMIC_LINKING
131   2: 00400710    4 OBJECT  GLOBAL  DEFAULT 16 _IO_stdin_used
132   3: 00420010    0 OBJECT  GLOBAL  DEFAULT 21 __RLD_MAP
133   4: 00400650   84 FUNC    GLOBAL  DEFAULT 13 main
134   5: 00000000    0 NOTYPE  WEAK    DEFAULT UND _ITM_registerTMC[...]
135   6: 00000000    0 FUNC    WEAK    DEFAULT UND __gmon_start__
136   7: 004006c0    0 FUNC    GLOBAL  DEFAULT UND puts@GLIBC_2.0 (3)
137   8: 00000000    0 NOTYPE  WEAK    DEFAULT UND _ITM_deregisterT[...]
138   9: 004006b0    0 FUNC    GLOBAL  DEFAULT UND _[...]@GLIBC_2.34 (2)
139
140 Symbol table '.symtab' contains 70 entries:
141   Num:   Value   Size Type   Bind   Vis     Ndx Name
142   0: 00000000    0 NOTYPE  LOCAL  DEFAULT UND
143   1: 00400194    0 SECTION LOCAL  DEFAULT 1 .interp
144   2: 004001a8    0 SECTION LOCAL  DEFAULT 2 .MIPS.abiflags
145   3: 004001c0    0 SECTION LOCAL  DEFAULT 3 .reginfo
146   4: 004001d8    0 SECTION LOCAL  DEFAULT 4 .note.gnu.build-id
147   5: 004001fc    0 SECTION LOCAL  DEFAULT 5 .note.ABI-tag
148   6: 0040021c    0 SECTION LOCAL  DEFAULT 6 .dynamic
149   7: 0040031c    0 SECTION LOCAL  DEFAULT 7 .hash
150   8: 00400358    0 SECTION LOCAL  DEFAULT 8 .dynsym
151   9: 004003f8    0 SECTION LOCAL  DEFAULT 9 .dynstr
152  10: 0040049e    0 SECTION LOCAL  DEFAULT 10 .gnu.version
153  11: 004004b4    0 SECTION LOCAL  DEFAULT 11 .gnu.version_r
154  12: 004004e4    0 SECTION LOCAL  DEFAULT 12 .init

```

155	13: 00400520	0	SECTION	LOCAL	DEFAULT	13 .text
156	14: 004006b0	0	SECTION	LOCAL	DEFAULT	14 .MIPS.stubs
157	15: 004006e0	0	SECTION	LOCAL	DEFAULT	15 .fini
158	16: 00400710	0	SECTION	LOCAL	DEFAULT	16 .rodata
159	17: 00400730	0	SECTION	LOCAL	DEFAULT	17 .eh_frame
160	18: 0041ffff8	0	SECTION	LOCAL	DEFAULT	18 .init_array
161	19: 0041ffffc	0	SECTION	LOCAL	DEFAULT	19 .fini_array
162	20: 00420000	0	SECTION	LOCAL	DEFAULT	20 .data
163	21: 00420010	0	SECTION	LOCAL	DEFAULT	21 .rld_map
164	22: 00420020	0	SECTION	LOCAL	DEFAULT	22 .got
165	23: 00420040	0	SECTION	LOCAL	DEFAULT	23 .sdata
166	24: 00420050	0	SECTION	LOCAL	DEFAULT	24 .bss
167	25: 00000000	0	SECTION	LOCAL	DEFAULT	25 .comment
168	26: 00000000	0	SECTION	LOCAL	DEFAULT	26 .pdr
169	27: 00000000	0	SECTION	LOCAL	DEFAULT	27 .gnu.attributes
170	28: 00000000	0	SECTION	LOCAL	DEFAULT	28 .mdebug.abi32
171	29: 00000000	0	FILE	LOCAL	DEFAULT	ABS crt1.o
172	30: 004001fc	32	OBJECT	LOCAL	DEFAULT	5 __abi_tag
173	31: 00400570	0	NOTYPE	LOCAL	DEFAULT	13 hlt
174	32: 00000000	0	FILE	LOCAL	DEFAULT	ABS crtstuff.c
175	33: 00400580	0	FUNC	LOCAL	DEFAULT	13 deregister_tm_clones
176	34: 004005b8	0	FUNC	LOCAL	DEFAULT	13 register_tm_clones
177	35: 00400604	0	FUNC	LOCAL	DEFAULT	13 __do_global_dtors_aux
178	36: 00420050	1	OBJECT	LOCAL	DEFAULT	24 completed.0
179	37: 0041ffffc	0	OBJECT	LOCAL	DEFAULT	19 __do_global_dtor[...]
180	38: 00400640	0	FUNC	LOCAL	DEFAULT	13 frame_dummy
181	39: 0041ffff8	0	OBJECT	LOCAL	DEFAULT	18 __frame_dummy_in[...]
182	40: 00000000	0	FILE	LOCAL	DEFAULT	ABS main.c
183	41: 00000000	0	FILE	LOCAL	DEFAULT	ABS crtstuff.c
184	42: 00400730	0	OBJECT	LOCAL	DEFAULT	17 __FRAME_END__
185	43: 00000000	0	FILE	LOCAL	DEFAULT	ABS
186	44: 004006b0	0	FUNC	LOCAL	DEFAULT	14 _MIPS_STUBS_
187	45: 0040021c	0	OBJECT	LOCAL	DEFAULT	6 _DYNAMIC
188	46: 00428010	0	NOTYPE	LOCAL	DEFAULT	22 _gp
189	47: 004006b0	0	FUNC	GLOBAL	DEFAULT	UND __libc_start_mai[...]
190	48: 00000001	0	SECTION	GLOBAL	DEFAULT	ABS _DYNAMIC_LINKING
191	49: 00000000	0	NOTYPE	WEAK	DEFAULT	UND _ITM_deregisterT[...]
192	50: 00420000	0	NOTYPE	WEAK	DEFAULT	20 data_start
193	51: 00420044	0	NOTYPE	GLOBAL	DEFAULT	23 _edata
194	52: 004006e0	0	FUNC	GLOBAL	HIDDEN	15 _fini
195	53: 00420000	0	NOTYPE	GLOBAL	DEFAULT	20 __data_start
196	54: 004006c0	0	FUNC	GLOBAL	DEFAULT	UND puts@GLIBC_2.0
197	55: 00000000	0	FUNC	WEAK	DEFAULT	UND __gmon_start__
198	56: 00420040	0	OBJECT	GLOBAL	HIDDEN	23 __dso_handle
199	57: 00400710	4	OBJECT	GLOBAL	DEFAULT	16 _IO_stdin_used
200	58: 00420000	0	NOTYPE	GLOBAL	DEFAULT	20 _fdata
201	59: 00420010	0	OBJECT	GLOBAL	DEFAULT	21 __RLD_MAP
202	60: 00420060	0	NOTYPE	GLOBAL	DEFAULT	24 _end
203	61: 00420050	0	NOTYPE	GLOBAL	DEFAULT	24 __bss_start
204	62: 00400650	84	FUNC	GLOBAL	DEFAULT	13 main
205	63: 00400520	0	FUNC	GLOBAL	DEFAULT	13 __start
206	64: 00400520	0	NOTYPE	GLOBAL	DEFAULT	13 _ftext
207	65: 00420014	0	OBJECT	GLOBAL	HIDDEN	21 __TMC_END__
208	66: 00420020	0	OBJECT	GLOBAL	HIDDEN	22 _GLOBAL_OFFSET_TABLE_
209	67: 00420050	0	NOTYPE	GLOBAL	DEFAULT	24 _fbss
210	68: 00000000	0	NOTYPE	WEAK	DEFAULT	UND _ITM_registerTMC[...]
211	69: 004004e4	0	FUNC	GLOBAL	HIDDEN	12 _init
212						

```

213 Histogram for bucket list length (total of 3 buckets):
214   Length  Number      % of total  Coverage
215       0  0          ( 0.0%)
216       1  1          ( 33.3%)   11.1%
217       2  1          ( 33.3%)   33.3%
218       3  0          ( 0.0%)   33.3%
219       4  0          ( 0.0%)   33.3%
220       5  0          ( 0.0%)   33.3%
221       6  1          ( 33.3%)  100.0%
222
223 Version symbols section '.gnu.version' contains 10 entries:
224   Addr: 0x000000000040049e  offset: 0x0000049e  Link: 8 (.dynsym)
225   000:   0 (*本地*)       1 (*全局*)       1 (*全局*)       1 (*全局*)
226   004:   1 (*全局*)       1 (*全局*)       1 (*全局*)       3 (GLIBC_2.0)
227   008:   1 (*全局*)       2 (GLIBC_2.34)
228
229 Version needs section '.gnu.version_r' contains 1 entry:
230   Addr: 0x00000000004004b4  offset: 0x000004b4  Link: 9 (.dynstr)
231   000000: Version: 1 文件: libc.so.6 计数: 2
232   0x0010:   Name: GLIBC_2.0 标志: 无 版本: 3
233   0x0020:   Name: GLIBC_2.34 标志: 无 版本: 2
234
235 Displaying notes found in: .note.gnu.build-id
236   所有者          Data size  Description
237   GNU              0x00000014      NT_GNU_BUILD_ID (unique build ID
bitstring)
238   Build ID: e15deb55690859893a6ca4c908962cd0bf28de67
239
240 Displaying notes found in: .note.ABI-tag
241   所有者          Data size  Description
242   GNU              0x00000010      NT_GNU_ABI_TAG (ABI version tag)
243   OS: Linux, ABI: 3.2.0
244 Attribute Section: gnu
245 File Attributes
246   Tag_GNU_MIPS_ABI_FP: Hard float (32-bit CPU, Any FPU)
247
248 MIPS ABI Flags Version: 0
249
250 ISA: MIPS32r2
251 GPR size: 32
252 CPR1 size: 32
253 CPR2 size: 0
254 FP ABI: Hard float (32-bit CPU, Any FPU)
255 ISA Extension: 无
256 ASEs:
257   无
258 FLAGS 1: 00000000
259 FLAGS 2: 00000000
260
261 Primary GOT:
262   Canonical gp value: 00428010
263
264 Reserved entries:
265   住址      访问  Initial Purpose
266   00420020 -32752(gp) 00000000 Lazy resolver
267   00420024 -32748(gp) 80000000 Module pointer (GNU extension)
268
269 Local entries:

```

	住址	访问	Initial				
270							
271	00420028	-32744(gp)	00400650				
272							
273	Global entries:						
274	住址	访问	Initial	Sym.val.	类型	Ndx	名字
275	0042002c	-32740(gp)	00000000	00000000	NOTYPE	UND	
	_ITM_registerTMCloneTable						
276	00420030	-32736(gp)	00000000	00000000	FUNC	UND	__gmon_start__
277	00420034	-32732(gp)	004006c0	004006c0	FUNC	UND	puts
278	00420038	-32728(gp)	00000000	00000000	NOTYPE	UND	
	_ITM_deregisterTMCloneTable						
279	0042003c	-32724(gp)	004006b0	004006b0	FUNC	UND	__libc_start_main

objdump

执行 `mips-linux-gnu-objdump -S main_mips` 得到以下结果

```

1  main_mips:      文件格式 elf32-tradbigmips
2
3
4  Disassembly of section .init:
5
6  004004e4 <_init>:
7      4004e4:      3c1c0002      lui        gp,0x2
8      4004e8:      279c7b2c      addiu     gp,gp,31532
9      4004ec:      0399e021      addu      gp,gp,t9
10     4004f0:      27bdffe0      addiu     sp,sp,-32
11     4004f4:      afbc0010      sw        gp,16(sp)
12     4004f8:      afbf001c      sw        ra,28(sp)
13     4004fc:      8f828020      lw        v0,-32736(gp)
14     400500:      10400004      beqz      v0,400514 <_init+0x30>
15     400504:      00000000      nop
16     400508:      8f998020      lw        t9,-32736(gp)
17     40050c:      0320f809      jalr      t9
18     400510:      00000000      nop
19     400514:      8fbf001c      lw        ra,28(sp)
20     400518:      03e00008      jr        ra
21     40051c:      27bd0020      addiu     sp,sp,32
22
23  Disassembly of section .text:
24
25  00400520 <__start>:
26     400520:      03e00025      move      zero,ra
27     400524:      04110001      bal      40052c <__start+0xc>
28     400528:      00000000      nop
29     40052c:      3c1c0002      lui        gp,0x2
30     400530:      279c7ae4      addiu     gp,gp,31460
31     400534:      039fe021      addu      gp,gp,ra
32     400538:      0000f825      move      ra,zero
33     40053c:      8f848018      lw        a0,-32744(gp)
34     400540:      8fa50000      lw        a1,0(sp)
35     400544:      27a60004      addiu     a2,sp,4
36     400548:      2401fff8      li        at,-8
37     40054c:      03a1e824      and       sp,sp,at
38     400550:      27bdffe0      addiu     sp,sp,-32
39     400554:      00003825      move      a3,zero
40     400558:      afa00010      sw        zero,16(sp)

```

```

41 40055c: afa20014 sw v0,20(sp)
42 400560: afbd0018 sw sp,24(sp)
43 400564: 8f99802c lw t9,-32724(gp)
44 400568: 0320f809 jalr t9
45 40056c: 00000000 nop
46
47 00400570 <hlt>:
48 400570: 1000ffff b 400570 <hlt>
49 400574: 00000000 nop
50 ...
51
52 00400580 <deregister_tm_clones>:
53 400580: 3c040042 lui a0,0x42
54 400584: 3c020042 lui v0,0x42
55 400588: 24840014 addiu a0,a0,20
56 40058c: 24420014 addiu v0,v0,20
57 400590: 10440007 beq v0,a0,4005b0
58 <deregister_tm_clones+0x30>
59 400594: 3c1c0043 lui gp,0x43
60 400598: 279c8010 addiu gp,gp,-32752
61 40059c: 8f998028 lw t9,-32728(gp)
62 4005a0: 13200003 beqz t9,4005b0
63 <deregister_tm_clones+0x30>
64 4005a4: 00000000 nop
65 4005a8: 03200008 jr t9
66 4005ac: 00000000 nop
67 4005b0: 03e00008 jr ra
68 4005b4: 00000000 nop
69
70 004005b8 <register_tm_clones>:
71 4005b8: 3c040042 lui a0,0x42
72 4005bc: 3c020042 lui v0,0x42
73 4005c0: 24840014 addiu a0,a0,20
74 4005c4: 24450014 addiu a1,v0,20
75 4005c8: 00a42823 subu a1,a1,a0
76 4005cc: 00051083 sra v0,a1,0x2
77 4005d0: 00052fc2 srl a1,a1,0x1f
78 4005d4: 00a22821 addu a1,a1,v0
79 4005d8: 00052843 sra a1,a1,0x1
80 4005dc: 10a00007 beqz a1,4005fc
81 <register_tm_clones+0x44>
82 4005e0: 3c1c0043 lui gp,0x43
83 4005e4: 279c8010 addiu gp,gp,-32752
84 4005e8: 8f99801c lw t9,-32740(gp)
85 4005ec: 13200003 beqz t9,4005fc
86 <register_tm_clones+0x44>
87 4005f0: 00000000 nop
88 4005f4: 03200008 jr t9
89 4005f8: 00000000 nop
90 4005fc: 03e00008 jr ra
91 400600: 00000000 nop
92
93 00400604 <__do_global_dtors_aux>:
94 400604: 27bdf0e0 addiu sp,sp,-32
95 400608: afb00018 sw s0,24(sp)
96 40060c: 3c100042 lui s0,0x42
97 400610: afbf001c sw ra,28(sp)
98 400614: 92020050 lbu v0,80(s0)

```



```

95      400618:      14400006      bnez      v0,400634
      <__do_global_dtors_aux+0x30>
96      40061c:      8fbf001c      lw        ra,28(sp)
97      400620:      0c100160      jal       400580 <deregister_tm_clones>
98      400624:      00000000      nop
99      400628:      24020001      li        v0,1
100     40062c:      a2020050      sb        v0,80(s0)
101     400630:      8fbf001c      lw        ra,28(sp)
102     400634:      8fb00018      lw        s0,24(sp)
103     400638:      03e00008      jr        ra
104     40063c:      27bd0020      addiu     sp,sp,32
105
106     00400640 <frame_dummy>:
107     400640:      0810016e      j         4005b8 <register_tm_clones>
108     400644:      00000000      nop
109     ...
110
111     00400650 <main>:
112     400650:      27bdffe0      addiu     sp,sp,-32
113     400654:      afbf001c      sw        ra,28(sp)
114     400658:      afbe0018      sw        s8,24(sp)
115     40065c:      03a0f025      move     s8,sp
116     400660:      3c1c0043      lui      gp,0x43
117     400664:      279c8010      addiu     gp,gp,-32752
118     400668:      afbc0010      sw        gp,16(sp)
119     40066c:      3c020040      lui      v0,0x40
120     400670:      24440720      addiu     a0,v0,1824
121     400674:      8f828024      lw        v0,-32732(gp)
122     400678:      0040c825      move     t9,v0
123     40067c:      0320f809      jalr     t9
124     400680:      00000000      nop
125     400684:      8fdc0010      lw        gp,16(s8)
126     400688:      00001025      move     v0,zero
127     40068c:      03c0e825      move     sp,s8
128     400690:      8fbf001c      lw        ra,28(sp)
129     400694:      8fbe0018      lw        s8,24(sp)
130     400698:      27bd0020      addiu     sp,sp,32
131     40069c:      03e00008      jr        ra
132     4006a0:      00000000      nop
133     ...
134
135     Disassembly of section .MIPS.stubs:
136
137     004006b0 <_MIPS_STUBS_>:
138     4006b0:      8f998010      lw        t9,-32752(gp)
139     4006b4:      03e07825      move     t7,ra
140     4006b8:      0320f809      jalr     t9
141     4006bc:      24180009      li        t8,9
142     4006c0:      8f998010      lw        t9,-32752(gp)
143     4006c4:      03e07825      move     t7,ra
144     4006c8:      0320f809      jalr     t9
145     4006cc:      24180007      li        t8,7
146     ...
147
148     Disassembly of section .fini:
149
150     004006e0 <_fini>:
151     4006e0:      3c1c0002      lui      gp,0x2

```

152	4006e4:	279c7930	addiu	gp, gp, 31024
153	4006e8:	0399e021	addu	gp, gp, t9
154	4006ec:	27bdffe0	addiu	sp, sp, -32
155	4006f0:	afbc0010	sw	gp, 16(sp)
156	4006f4:	afbf001c	sw	ra, 28(sp)
157	4006f8:	8fbf001c	lw	ra, 28(sp)
158	4006fc:	03e00008	jr	ra
159	400700:	27bd0020	addiu	sp, sp, 32

三、传入objdump的参数

使用 `man objdump` 查看之后可知，需要调用其进行反汇编时：

```
1 objdump -D file # 对file反汇编，返回汇编程序
2 objdump -S file # 对file反汇编，同时显示机器码和汇编程序
```

Thinking 1.2

Thinking 1.2 思考下述问题：

- 尝试使用我们编写的 `readelf` 程序，解析之前在 `target` 目录下生成的内核 ELF 文件。
- 也许你会发现我们编写的 `readelf` 程序是不能解析 `readelf` 文件本身的，而我们刚才介绍的系统工具 `readelf` 则可以解析，这是为什么呢？（提示：尝试使用 `readelf -h`，并阅读 `tools/readelf` 目录下的 `Makefile`，观察 `readelf` 与 `hello` 的不同）

补全 `readelf.c` 文件时，使用的是32位的宏来计算ELF文件的地址，故不能对64位的ELF解析。

一、解析target下的内核

执行 `./tools/readelf/readelf target/mos` 后得到以下结果

```
1 0:0x0
2 1:0x80020000
3 2:0x800220a0
4 3:0x800220b8
5 4:0x800220d0
6 5:0x0
7 6:0x0
8 7:0x0
9 8:0x0
10 9:0x0
11 10:0x0
12 11:0x0
13 12:0x0
14 13:0x0
15 14:0x0
16 15:0x0
17 16:0x0
18 17:0x0
19 18:0x0
```

二、使用 readelf -h

在 `tools/readelf/` 目录下生成 `hello` 和 `readelf` 后。

执行 `readelf -h readelf` 得到

```
1 ELF 头:
2   Magic:    7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00
3   类别:                                ELF64
4   数据:                                2 补码, 小端序 (little endian)
5   Version:                                1 (current)
6   OS/ABI:                                UNIX - System V
7   ABI 版本:                                0
8   类型:                                DYN (Position-Independent Executable
file)
9   系统架构:                                Advanced Micro Devices x86-64
10  版本:                                0x1
11  入口点地址:                                0x1180
12  程序头起点:                                64 (bytes into file)
13  Start of section headers:                14488 (bytes into file)
14  标志:                                0x0
15  Size of this header:                    64 (bytes)
16  Size of program headers:                56 (bytes)
17  Number of program headers:                13
18  Size of section headers:                64 (bytes)
19  Number of section headers:                31
20  Section header string table index: 30
```

执行 `readelf -h hello` 得到

```
1 ELF 头:
2   Magic:    7f 45 4c 46 01 01 01 03 00 00 00 00 00 00 00
3   类别:                                ELF32
4   数据:                                2 补码, 小端序 (little endian)
5   Version:                                1 (current)
6   OS/ABI:                                UNIX - GNU
7   ABI 版本:                                0
8   类型:                                EXEC (可执行文件)
9   系统架构:                                Intel 80386
10  版本:                                0x1
11  入口点地址:                                0x8049750
12  程序头起点:                                52 (bytes into file)
13  Start of section headers:                707128 (bytes into file)
14  标志:                                0x0
15  Size of this header:                    52 (bytes)
16  Size of program headers:                32 (bytes)
17  Number of program headers:                8
18  Size of section headers:                40 (bytes)
19  Number of section headers:                30
20  Section header string table index: 29
```

可以发现, `readelf` 的类别是 `ELF64`, 而 `hello` 是 `ELF32`。猜测: 我们的 `readelf` 只能解析32位的 ELF文件, 故其不能解析作为64位文件的自身。

Thinking 1.3

Thinking 1.3 在理论课上我们了解到, MIPS 体系结构上电时, 启动入口地址为 `0xBFC00000`

(其实启动入口地址是根据具体型号而定的, 由硬件逻辑确定, 也有可能不是这个地址, 但一定是一个确定的地址), 但实验操作系统的内核入口并没有放在上电启动地址, 而是按照内存布局图放置。思考为什么这样放置内核还能保证内核入口被正确跳转到? (提示: 思考实验中启动过

程的两阶段分别由谁执行。)

启动的第一阶段是初始化硬件设备，在 ROM 中由 bootloader 执行，第二阶段在 RAM 中，初始化该阶段硬件设备，读取并载入内核，执行引导程序，因此启动入口地址未必是内核入口地址，而引导程序的存在会把内核载入内存并跳转，保证内核入口被正确跳转到。

具体方法是使用跳转指令，就像 exercise 1.3 中使用的 `jal mips_init` 类似。

练习题

Exercise 1.1

Exercise 1.1 阅读 tools/readelf 目录下的 elf.h、readelf.c 和 main.c 文件，并补全 readelf.c 中缺少的代码。readelf 函数需要输出 ELF 文件中所有节头中的地址信息，对于每个节头，输出格式为 "%d:0x%x\n"，其中的 %d 和 %x 分别代表序号和地址。正确完成 readelf.c 之后，在 tools/readelf 目录下执行 make 命令，即可生成可执行文件 readelf，它接受文件名作为参数，对 ELF 文件进行解析。可以执行 make hello 生成测试用的 ELF 文件 hello，然后运行 ./readelf hello 来测试 readelf。

```
1  /* Exercise 1.1: Your code here. (1/2) */
2      // 计算节头表起始位置 = 文件起始地址 + 节头表偏移
3      sh_table = (const char *)binary + ehdr->e_shoff;
4      // 获取节头表项数量和单个表项大小
5      sh_entry_count = ehdr->e_shnum;
6      sh_entry_size = ehdr->e_shentsize;
7
8  /* Exercise 1.1: Your code here. (2/2) */
9      // 计算当前节头的地址: 表起始地址 + 索引*表项大小
10     shdr = (const Elf32_Shdr *)((const char *)sh_table + i *
sh_entry_size);
11     // 获取该节在内存中的虚拟地址
12     addr = shdr->sh_addr;
```

这题本质就是计算地址偏移量，需要阅读头文件 elf.h 中的结构体内的数据结构来获取需要的对应信息。

```
1  // ELF头文件结构
2  typedef struct {
3      unsigned char e_ident[EI_NIDENT]; /* Magic number and other info */
4      Elf32_Half e_type; /* Object file type */
5      Elf32_Half e_machine; /* Architecture */
6      Elf32_Word e_version; /* Object file version */
7      Elf32_Addr e_entry; /* Entry point virtual address */
8      Elf32_Off e_phoff; /* Program header table file offset */
9      Elf32_Off e_shoff; /* Section header table file offset */
10     Elf32_Word e_flags; /* Processor-specific flags */
11     Elf32_Half e_ehsize; /* ELF header size in bytes */
12     Elf32_Half e_phentsize; /* Program header table entry size */
13     Elf32_Half e_phnum; /* Program header table entry count */
14     Elf32_Half e_shentsize; /* Section header table entry size */
15     Elf32_Half e_shnum; /* Section header table entry count */
16     Elf32_Half e_shstrndx; /* Section header string table index */
17 } Elf32_Ehdr;
18
19 // 节的结构
```

```

20 typedef struct {
21     Elf32_Word sh_name; /* Section name */
22     Elf32_Word sh_type; /* Section type */
23     Elf32_Word sh_flags; /* Section flags */
24     Elf32_Addr sh_addr; /* Section addr */
25     Elf32_Off sh_offset; /* Section offset */
26     Elf32_Word sh_size; /* Section size */
27     Elf32_Word sh_link; /* Section link */
28     Elf32_Word sh_info; /* Section extra info */
29     Elf32_Word sh_addralign; /* Section alignment */
30     Elf32_Word sh_entsize; /* Section entry size */
31 } Elf32_Shdr;

```

Exercise 1.2

Exercise 1.2 填写 kernel.ld 中空缺的部分，在 Lab1 中，只需要填补 .text、.data 和 .bss 节，将内核调整到正确的位置上即可。

```

1  SECTIONS {
2      . = 0x80020000; /* KERNBASE 的地址值可以在 include/mmu.h 中找到 */
3      .text : {
4          *(.text)
5      }
6      .data : {
7          *(.data)
8      }
9      bss_start = .;
10     .bss : {
11         *(.bss)
12     }
13     bss_end = .;
14     . = 0x80400000; /* 也即 KSTACKTOP 的值 */
15     end = . ;
16 }

```

.text 设置在 KERNBASE，剩下的接在其后面就行。

Exercise 1.3

Exercise 1.3 完成 init/start.S 中空缺的部分。设置栈指针，跳转到 mips_init 函数。执行命令 make run 运行仿真，或使用命令 make dbg_run 在调试模式下运行 QEMU。

```

1      /* hint: you can refer to the memory layout in include/mmu.h */
2      /* set up the kernel stack */
3      /* Exercise 1.3: Your code here. (1/2) */
4
5      li sp, 0x80400000 /* 可以在include/mmu.h中找到，即为 KSTACKTOP */
6
7      /* jump to mips_init */
8      /* Exercise 1.3: Your code here. (2/2) */
9
10     jal mips_init
11     nop

```

这里需要根据内存空间设置栈指针到 KSTACKTOP 的位置。跳转则直接跳即可。

这里需要注意mips架构中栈指针的方向：`.text` 位于 `KERNBASE` 之上，`stack` 则位于 `KSTACKTOP` 之下。也就是说 `KERNBASE` 到 `KSTACKTOP` 之间的空间存储了指令和栈，两者分别朝着不同方向增加。

Exercise 1.4

Exercise 1.4 阅读相关代码和下面对于函数规格的说明，补全 `lib/print.c` 中 `vprintfmt()` 函数中两处缺失的部分来实现字符输出。第一处缺失部分：找到 `%` 并分析输出格式；第二处 缺失部分：取出参数，输出格式串为 `%[flags][width][length]` 的情况。具体格式详见 `printk` 格式具体说明。

需要利用C语言来解析格式化字符，一些工具函数已经给出。

```
1 void vprintfmt(fmt_callback_t out, void *data, const char *fmt, va_list
  ap) {
2     char c;
3     const char *s;
4     long num;
5
6     int width;
7     int long_flag; // output is long (rather than int)
8     int neg_flag;  // output is negative
9     int ladjust;   // output is left-aligned
10    char padc;      // padding char
11
12    for (;;) {
13        /* scan for the next '%'
14        扫描下一个百分号 */
15        /* Exercise 1.4: Your code here. (1/8) */
16
17        const char *start = fmt;          // 记录当前指针位置
18        while (*fmt != '%' && *fmt) fmt++;  // fmt指针移动，直到找到百分号
19        为止
20
21        /* flush the string found so far
22        输出目前发现的字符串（即输出start到当前fmt之间的不含格式化字符的字符串） */
23        /* Exercise 1.4: Your code here. (2/8) */
24
25        if (fmt > start) {
26            out(data, start, fmt - start);
27        }
28
29        /* check "are we hitting the end?"
30        检查我们是否已经解析字符串完毕 */
31        /* Exercise 1.4: Your code here. (3/8) */
32
33        if (!*fmt) break;
34
35        /* we found a '%'
36        处理百分号 */
37        /* Exercise 1.4: Your code here. (4/8) */
38
39        fmt++;                          // 跳过百分号
40
41        /* check format flag
42        解析格式标志（除了neg_flag之外的四个） */
43        /* Exercise 1.4: Your code here. (5/8) */
44
45        // 标志初始化
```

```

45     ladjust = 0;
46     padc = ' ';
47     width = 0;
48     long_flag = 0;
49
50     // 处理对齐标志
51     if (*fmt == '-') {
52         ladjust = 1;
53         fmt++;
54     }
55
56     // 处理填充字符
57     if (*fmt == '0') {
58         if (!ladjust) {
59             padc = '0';
60         }
61         fmt++;
62     }
63
64     /* get width
65     获取宽度 */
66     /* Exercise 1.4: Your code here. (6/8) */
67
68     for ( ; *fmt >= '0' && *fmt <= '9'; fmt++ ) {
69         width = *fmt - '0' + width * 10;        // 计算宽度
70     }
71
72     /* check for long
73     处理long标志 */
74     /* Exercise 1.4: Your code here. (7/8) */
75
76     if (*fmt == 'l') {
77         long_flag = 1;
78         fmt++;
79     }
80
81     neg_flag = 0;
82     switch (*fmt) {
83     case 'b':
84         if (long_flag) {
85             num = va_arg(ap, long int);
86         } else {
87             num = va_arg(ap, int);
88         }
89         print_num(out, data, num, 2, 0, width, ladjust, padc, 0);
90         break;
91
92     case 'd':
93     case 'D':
94         if (long_flag) {
95             num = va_arg(ap, long int);
96         } else {
97             num = va_arg(ap, int);
98         }
99
100        /*
101        * Refer to other parts (case 'b', case 'o', etc.) and func
'print_num' to

```

```

102         * complete this part. Think the differences between case 'd'
and the
103         * others. (hint: 'neg_flag').
104         */
105         /* Exercise 1.4: Your code here. (8/8) */
106
107         // 处理负号
108         if (num < 0) {
109             num = -num;
110             neg_flag = 1;
111         }
112
113         // 输出
114         print_num(out, data, num, 10, neg_flag, width, ladjust, padc,
0);
115
116         break;
117
118     case 'o':
119     case 'O':
120         if (!long_flag) {
121             num = va_arg(ap, long int);
122         } else {
123             num = va_arg(ap, int);
124         }
125         print_num(out, data, num, 8, 0, width, ladjust, padc, 0);
126         break;
127
128     case 'u':
129     case 'U':
130         if (!long_flag) {
131             num = va_arg(ap, long int);
132         } else {
133             num = va_arg(ap, int);
134         }
135         print_num(out, data, num, 10, 0, width, ladjust, padc, 0);
136         break;
137
138     case 'x':
139         if (!long_flag) {
140             num = va_arg(ap, long int);
141         } else {
142             num = va_arg(ap, int);
143         }
144         print_num(out, data, num, 16, 0, width, ladjust, padc, 0);
145         break;
146
147     case 'X':
148         if (!long_flag) {
149             num = va_arg(ap, long int);
150         } else {
151             num = va_arg(ap, int);
152         }
153         print_num(out, data, num, 16, 0, width, ladjust, padc, 1);
154         break;
155
156     case 'c':
157         c = (char)va_arg(ap, int);

```



```
158         print_char(out, data, c, width, ladjust);
159         break;
160
161     case 's':
162         s = (char *)va_arg(ap, char *);
163         print_str(out, data, s, width, ladjust);
164         break;
165
166     case '\\0':
167         fmt--;
168         break;
169
170     default:
171         /* output this char as it is */
172         out(data, fmt, 1);
173     }
174     fmt++;
175 }
176 }
```

难点

主要难点是对实验代码的阅读，目前还是有很多地方没看懂，需要多看，理解各部分的联系。